Deliverable 2: The Code

What's in the repository, and how to run it

Mills-Operation · AI Reliability Copilot for Hot Mill Operations

The Code

Everything written for this lives in one repository. This deliverable walks through what is actually in it, folder by folder and the key files inside each, then gives the exact commands to run it. The narrative version of why things are built this way is in the architecture and stack justification deliverables; this is the map of where that thinking actually landed in code.

app/, the backend and the model

`app/features.py` and `app/labels.py` turn raw readings into what the model actually trains on: rolling mean, standard deviation, and rate of change per signal, plus time to and since the nearest failure so a row can be labeled clean normal or not. `app/detector.py` holds the trained LocalOutlierFactor model and the independent deviation backstop next to it. `app/evaluate.py` runs the whole training and scoring pipeline end to end and prints the real detection numbers.

`app/api.py` is the FastAPI backend, and everything else in this folder either feeds it or is called by it. `app/live_feed.py` is the background task that ticks once a real minute and keeps the live data moving. `app/historical.py` keeps the rolling 45 day scored window that backs the date range filter, cached to disk since scoring the full set takes too long to do inside a request. `app/simulator.py` and `app/signal_profile.py` back the degradation simulator, the correlated signal curve and the timed ramp toward failure. `app/copilot.py` is the AI layer, both the single stand explanation and the fleet wide chat, and `app/fleet_tools.py` is the six read only functions the fleet chat calls instead of using a vector database. `app/model_store.py` and `app/timeseries_utils.py` are small shared helpers (loading the trained model once, shaping alert bands for the frontend) used by more than one of the files above.

web/, the React and TypeScript frontend

`web/src/App.tsx` is the shell: the three tab switcher, the header, the polling loops that keep the fleet and the selected stand's chart data current. Everything else in `web/src/components/` is one piece of a screen: `FleetStrip` and `StandCard` render the landing grid, `SignalChart` and `TimeRangeFilter` are the per stand chart and its date range control, `SimulatorPage` and `FleetCopilotPage` are the other two tabs, `CurrentReadingsPanel` and `MiniFleetPanel` are the detail view's sidebar and fleet aware panel, `RollStandIcon` and `SoundToggle` are the animated stand icon and the draggable mute button for alert sound. `web/src/lib/alertSound.ts` handles the siren and spoken alert, `web/src/lib/markdown.tsx` renders the copilot's **bold** markers as real bold instead of showing literal asterisks. `web/src/api.ts` and `web/src/types.ts` are the one place every backend call and its response shape are defined, so the rest of the frontend never talks to `fetch` directly.

data/, tests/, docs/, notebooks/, scripts/

`data/generate_synthetic_data.py` is the seeded generator that produces the training dataset, described in full in the data strategy deliverable. `tests/dashboard.spec.js` is the Playwright suite covering the core flows, wired into `.github/workflows/ci.yml` so it runs on every push. `docs/` holds this project's actual design history, not retrofitted after the fact, including a plain English writeup of how the AI copilot works (`docs/copilot-how-it-works.md`). `notebooks/model_comparison.ipynb` is the real bake off that picked LocalOutlierFactor over four alternatives.

`scripts/` holds the one off utilities used to package these deliverables themselves (screenshot capture, diagram rendering, PDF generation), kept out of the main app since they are packaging tools, not part of the running product.

How to run it

```
# Python backend
pip install -r requirements.txt

# 1. Generate the synthetic sensor dataset (deterministic, about 389K rows, a few seconds)
python data/generate_synthetic_data.py

# 2. Train the anomaly detector and score the held out test window
python -m app.evaluate

# 3. Start the backend API (also starts the live feed in the background)
uvicorn app.api:app --reload --port 8000
```

```
# React frontend, in a second terminal
cd web
npm install
npm run dev
```

Open `http://localhost:5173`. Without an `ANTHROPIC_API_KEY` in `.env`, both AI surfaces still work, they fall back to a deterministic, data grounded response instead of a live model call, so the console never breaks because of a missing key.

How to test it

```
npm install
npx playwright install --with-deps chromium
npx playwright test
```

Playwright starts both the backend and the frontend itself, so just make sure the two setup commands above have already been run once. This runs automatically on every push through GitHub Actions.